AI MINI PROJECT REPORT

PARKSMART AI

Smart Parking Rental System using Artificial Intelligence and Machine Learning

Submitted By

Student Name: _____

B.Tech Information Technology

Priyadarshini College of Engineering, Nagpur

Academic Year: 2026–27

AI • MACHINE LEARNING • SMART MOBILITY

1. Aim

To design and implement an AI/ML-based Smart Parking Rental System that estimates hourly parking prices and demonstrates how unused private or apartment parking spaces could be offered for short-term rental.

2. Problem Definition

In large cities, drivers often spend time searching for parking while some apartment parking spaces remain unused when their owners are away. A rental platform could connect space owners and drivers. Price estimation is one useful AI component; real availability and bookings require verified owner input and a live scheduling system.

3. Abstract

ParkSmart AI is an AI/ML mini-project developed with Python, Flask, Pandas and Scikit-learn. It estimates hourly rental price using area, parking type, security, covered status, distance to the city centre, demand level, start hour and rental duration. A Random Forest Regressor is trained on an included synthetic dataset. The Flask dashboard accepts selected parking characteristics and displays an estimated hourly rate and total rental cost. This academic prototype does not provide real-time availability, payments or actual bookings.

4. Objectives

• Develop a web-based smart parking rental prototype.

• Create a structured demonstration dataset of parking features and indicative prices.

• Preprocess categorical features using One-Hot Encoding.

• Train a Random Forest regression model.

• Estimate hourly price and total rental cost.

• Display evaluation metrics and sample parking listings.

• Document possible extensions to a real marketplace.

5. Introduction

Parking demand can be high around commercial areas, universities, hospitals and dense residential neighbourhoods. Some private parking bays may be unused for several hours. ParkSmart AI demonstrates a possible sharing-economy solution. Machine learning estimates an indicative price, while the web interface explains the rental flow. Real deployment would require owner consent, verified listings, a reliable availability calendar, secure payments and safety controls.

6. Technologies Used

Component	Technology
Programming	Python 3
Web framework	Flask
Data processing	Pandas
Machine learning	Scikit-learn
Algorithm	Random Forest Regression
Frontend	HTML and CSS
Dataset	CSV synthetic data
IDE	Visual Studio Code / Python IDE

7. AI / Machine Learning Technique

7.1 Supervised Regression

The model learns a relationship between labelled input features and a numeric target, hourly_price_inr, then predicts a price for a new combination of features.

7.2 Random Forest Regressor

Random Forest builds multiple decision trees and averages predictions. It can represent non-linear relationships in structured data.

7.3 One-Hot Encoding

Categorical values such as area and parking type are converted into numerical indicator columns within a Scikit-learn pipeline.

8. Dataset and Features

The included CSV contains 800 synthetic records and 10 columns. The data is generated for demonstration and does not represent observed market prices.

Feature	Type	Description
area	Categorical	Area category: Central, Residential, Business District, University Area or Suburban
parking_type	Categorical	Apartment, Commercial or Open Lot
security / covered	Categorical	Whether security or a roof is provided
distance_to_centre_km	Numeric	Distance from city centre
demand_level	Numeric	Demonstration rating from 1 to 5
hour_of_day	Numeric	Start hour 0–23
duration_hours	Numeric	Requested rental duration
hourly_price_inr	Target	Synthetic hourly price in rupees

9. System Requirements

• Laptop/desktop; 4 GB RAM or more recommended.

• Python 3.10 or newer and pip.

• Modern web browser.

• Packages listed in requirements.txt.

10. Input Parameters

• Area category and parking type

• Security and covered-space status

• Distance to city centre

• Demand level (1–5)

• Start hour (0–23)

• Rental duration in hours

11. Target Output

• Predicted hourly rental price in INR.

• Estimated total = hourly estimate × selected duration.

• Test-set Mean Absolute Error (MAE) and R^2 score.

These are estimates based on synthetic data, not real quotes or guaranteed prices.

12. Working / Methodology

• Load parking_data.csv using Pandas.

• Select input features and hourly_price_inr target.

• One-hot encode categorical columns.

• Split data into training and test sets (80:20).

• Train a Random Forest Regressor with 180 trees.

• Evaluate using MAE and R^2 on held-out data.

• Receive details from the Flask form.

• Predict hourly price, calculate total cost and show results.

13. Algorithm

1. Start Flask application.

2. Read dataset and select features/target.

3. Build preprocessing and regression pipeline.

4. Split data into training and testing subsets.

5. Train the Random Forest Regressor.

6. Calculate evaluation metrics.

7. Receive parking characteristics from the web form.

8. Predict hourly price and total cost.

9. Display results and sample listings.

10. Wait for the next request.

14. Program

Core model configuration:

```
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

preprocessor = ColumnTransformer([
    ("categorical", OneHotEncoder(handle_unknown="ignore"),
     categorical_features)
], remainder="passthrough")

model = Pipeline([
    ("preprocess", preprocessor),
    ("regressor", RandomForestRegressor(
        n_estimators=180, max_depth=12, random_state=42))
])
```

The complete implementation is in app.py. The interface is in templates/index.html and static/style.css; the dataset is parking_data.csv.

15. Sample Output

Example input: Central area; Apartment parking; security and cover enabled; 3 km from centre; demand level 3; start hour 18; duration 2 hours.

The exact predicted value is calculated by the included model when the application runs, so it is not hard-coded here.

16. Application Dashboard Output

The dashboard contains the estimator form, hourly and total price estimate, test-set evaluation metrics and example records from the sample dataset. The sample listings are not live spaces and do not show real availability.

17. Model Evaluation

The model is evaluated on a held-out 20% test set. MAE indicates the average absolute difference between predicted and synthetic target prices in rupees; lower is better. R^2 describes how much target variation is explained by the model. Because the data is synthetic, the metrics do not establish real-world pricing accuracy.

18. Advantages

- Simple browser-based interface.
- Demonstrates a supervised ML pipeline.
- Reproducible dataset and source code included.
- Estimates hourly and total rental cost.
- Supports categorical and numeric inputs.
- Displays MAE and R^2.
- Can be extended with bookings and maps.

19. Limitations

- Synthetic data, not real market rates.
- No login, owner verification or apartment permission workflow.
- No real-time calendar or booking conflict prevention.
- No payment gateway.
- No maps or live occupancy sensors.
- Model estimates may not generalise to real locations.
- Development server is not for production use.

20. Future Scope

- Collect consented, anonymised real price and occupancy data.
- Add owner accounts and verified listings.
- Implement availability calendars and booking confirmation.
- Integrate maps and location search.
- Add secure payments, cancellations and notifications.
- Compare Random Forest with other regressors.
- Deploy with production server and security/privacy reviews.

21. Screenshots / Output

After running the application, capture (1) the initial dashboard, (2) the estimator after entering details, and (3) the displayed prediction. Insert your own screenshots before submission; do not label mock images as execution evidence.

22. Final Output Evidence

Project files include app.py, parking_data.csv, templates/index.html, static/style.css, requirements.txt, README.md and .gitignore. Run the application and capture its actual output for this page.

How to run the application

1. Open the project folder in VS Code.
2. Open Terminal in that folder.
3. Run: py -m venv .venv
4. Run: .venv\Scripts\activate
5. Run: pip install -r requirements.txt
6. Run: python app.py
7. Open: http://127.0.0.1:5000

GitHub repository link: Paste the actual repository URL here after creating and uploading the repository. Example format: https://github.com/YOUR-USERNAME/ParkSmart-AI

Before submission: Replace the student name, add your own screenshots and paste the real GitHub URL.